Group-15 (Aangi Jignesh Shah, Dhruvee Parasbhai Sheth)

1. Discuss three major complications that concurrent processing adds to an operating system.
2. When a process creates a new process using the fork() operation, which of the following states is shared between the parent process and the child process?
3. Describe the /proc filesystem in Unix for managing processes.

Answer 1:

1. Race Conditions

A big complication in concurrent processing is the risk of race conditions. This happens when two or more processes try to use the same data or resource at the same time, and the final result depends on who gets there first. For example, if two processes both try to update a bank account balance at the same time, the result might be wrong if they don't update in the correct order. The operating system must provide tools like locks or semaphores to make sure only one process uses shared data at a time. Without these, the system can produce incorrect results or corrupted data.

2. Deadlocks

Another complication is deadlock, which happens when processes get stuck waiting for each other's resources. For example, one process may be holding a printer and waiting for a file, while another process holds the file and waits for the printer. In this situation, neither can continue. The operating system needs ways to prevent or handle deadlocks so that the system does not freeze. It also has to make sure that resources are given out fairly, so no process waits forever while others keep getting what they want.

3. Hard to Debug and Test

Concurrent systems are also much harder to test and debug compared to single-process systems. The order in which processes run can change every time, depending on the CPU and system timing. This makes some errors appear only rarely, and they may be very hard to reproduce. For example, a program might work fine most of the time but suddenly fail when two processes happen to run at the same moment. Because of this, developers and operating systems need special tools and careful design to catch these timing-related errors.

Answer 2:

When a process calls fork(), the operating system makes a new child process that is a copy of the parent. The child gets its own memory, stack, and registers, so changes made by the child do not affect the parent's variables or instructions. In this way, both processes can run independently after the fork.

What they do share are the parent's open files. This means that if the parent had already opened a file before calling fork, the child can also read from or write to that same file. Both processes also share the file offset, so if one moves forward in the file, the other will see that change.

Answer 3:

The /proc filesystem in Unix is a special virtual file system that gives information about the system and running processes. It doesn't hold real files on disk instead, the kernel creates the contents whenever a file is read, so the data is always up to date.

Inside /proc, every process has its own directory named by its process ID (PID), like /proc/1234/, which contains details about that process its state, memory usage, open files, and more. There are also system-wide files such as /proc/cpuinfo for processor details and /proc/meminfo for memory status. Many tools like ps and top rely on /proc to show process and system information. This makes /proc a bridge between the kernel and users, helping in monitoring and managing processes easily.